HDCSE4006 – Object Oriented Programming

Kirabage Malindu Vikash Indumina

Cardiff student number: st20312206

ICBT Student number: ST/HDCSE/CMU/49/11

Assignment number 10- Object Oriented Programming

\

Acknowledgement

I would like to sincerely acknowledge all of the parties involved in the success of this work. To my lecturer, Mr. Deloosha Abeysooriya, I offer my fullest appreciation for your supervision, motivation, advice, suggestions, and wisdom provided to me during the development of my assignment. I am truly grateful for all of the invaluable help and support you provided to me as I developed my understanding of Object-Oriented Programming. I acknowledge ICBT southern campus for providing opportunities and resources that would have limited the success of my project. My utmost gratitude goes to my family and their immense patience with me and motivation to finish/have success with my work. I would like to thank my batch mates, who provided useful suggestions and resources as well as just a little encouragement during times of hardship during this work. It was a pleasure and privilege to work with such amazing people. Thank you to all of the parties mentioned above; it was the collective efforts of all of the people involved that made finishing this work possible and to succeed.

Executive summary

This report gives a detailed development of The Sweet Cupcake Shop's automated management system, which will simplify the operations of the bakery, using the principles of object-oriented programming. The system uses role-based access control where Manager and Cashier are given different functionalities, so that there is a secure and efficient workflow management. The application created using Java (NetBeans IDE) and Swing GUI components and MS Access database, incorporates the basic concepts of OOP such as encapsulation, inheritance, abstraction, and polymorphism.

The development process includes three main tasks: UML modeling using Use Case, Class, and the Sequence diagrams, which illustrate the system structure and interaction; implementation of the main functionalities, such as user authentication, inventory management, sales processing, and transaction logging; and the creation of the elaborate user manual with operational instructions.

The system is effective in meeting the business requirements by dividing the operational and administrative tasks, ensuring the integrity of the data in the proper encapsulation, and facilitating the scalability of the code structure in terms of inheritance hierarchies. This solution can be used to illustrate how the OOP concepts of theory can be put into practice in the development of business management software that is applicable in the real world.

Table of Content

Table of Contents

List of figures

List of tables

No tables.

List of abbreviations

No abbreviations

Introduction

To react to escalating consumer demands and operational complexities, organizations are rapidly evolving to embrace digital transformation within today's functionality-focused business environment. "The Little Book Haven" is a new bookstore that provides a broad selection of literature from classic novels to recently published bestsellers in several genres (e.g., fiction, non-fiction, mystery). In order to remain competitive and maintain an exceptional customer experience, the bookstore will be changing from manually performing operations to using an automated transaction system.

The project that is intended will provide a completed, fully-automated system for processing all of the transactions that occur within the operational guidelines and procedures of The Little Book Haven’s business operation. The system consists of two levels of end-users: 1) the cashier, who will need to view book details, add new inventory items and search for books by category; and 2) the manager, who can perform all the cashier functions listed above as well as manage user accounts.

The Objective of this project is to provide a complete object-oriented programming (OOP) solution for The Little Book Haven's automated transaction system. The project will provide a finished and fully designed object-oriented programming (OOP) solution, and will include complete designing using Unified Modelling Language (UML) design diagrams, which will consist of Use Case diagrams, Class diagrams, and Sequence diagrams, as well as complete implementing the OOP solution (i.e., supporting all the principles of OOP: Recursion, Abstraction, Inheritance, Encapsulation, Polymorphism).

The Little Book Haven automated transaction system is expected to provide tremendous benefits to The Little Book Haven through increased operational efficiency, improved inventory control, and ultimately enhanced customer service.

Task 01

Use Case Diagram

The use case diagram is one of the most important tools used within Unified Modelling Language (UML) to model the functions of a system from the actors that interact with the system to create the functions of that system (i.e., interact with the system to complete the functions supported by the system) (Booch, et al.). Use Case Diagrams provide an overview from an external perspective to explain what the system does but not how it does it. Capturing the actors' external behaviours and documenting user requirements early are critical to defining and communicating user needs during the earliest development phases of software systems (Fowler).

Each Use Case Diagram has two major components:

1) Actors (Objects Management Group), which represent individuals, hardware, or systems that interact with the system(s); and

2) Use Cases (Arlow and Neustadt), which represent the various sets of actions and events that accomplish a specific outcome from the operator's point of view.

The primary use of Use Case Diagrams is to determine the system boundaries of a system and align the core functionality of the system with the actors who are affected (Fowler). Therefore, proper Use Case Diagrams assist in keeping the focus of development on the end-users.

The symbols used to create Use Case Diagrams are illustrated below.

NO	FIGURE	NAME	EXPLANATION
1		Actor	Functionality provided by the system as units that exchange messages between units or actors, is usually expressed using the verb at the beginning of the phrase name use case.
2		Use Case	People, processes, or other systems that interact with information systems that will be created outside the information system that will be created themselves.
3		Association	Communication between actors and use cases that participate in use cases has interaction with actors.
4		Include	Additional use case relations to a use case and use case that are added require this use case to carry out its functions.
5		Extend	Additional use case relations to a use case and use case that are added can stand alone even without additional use cases.

Figure 1

The following is a Use Case Diagram that has been developed for the proposed system to be designed/developed at The Little Book Haven business.

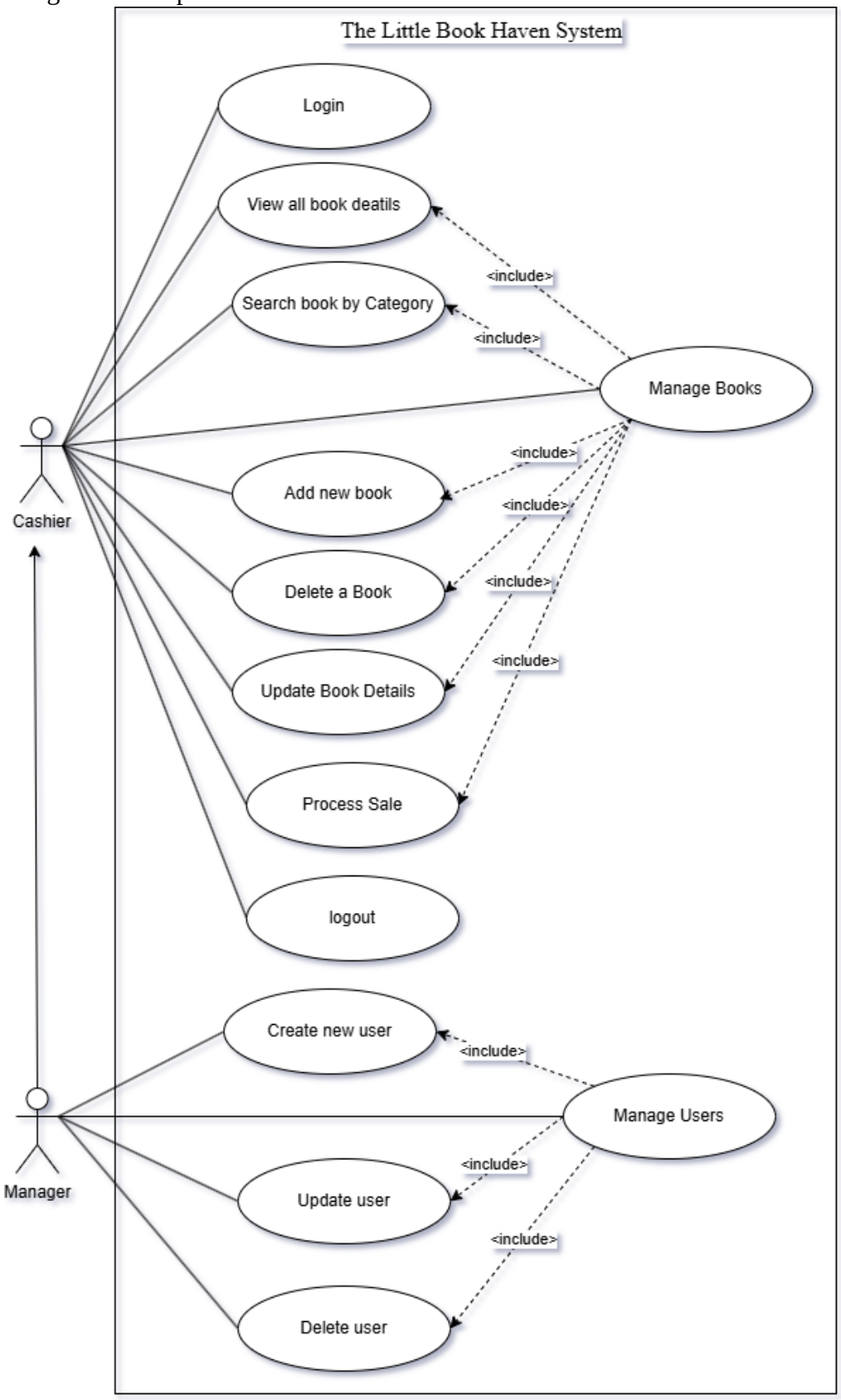


Figure 2

Key assumption made after analysing the Use case Diagram:

1. Actor Hierarchies and Permission Inheritance

The generalization arrow (solid with an open arrowhead) to connect the Cashier and Manager illustrates that the Manager is a more specific version of the Cashier. It is assumed that the Manager inherits all the Cashier functions (Processing Sales, Searching Books), and does not need direct association links to those particular use cases.

Role Separation: It is assumed that Manage Users (creating, updating and deleting staff accounts) is a restricted administrative area that a regular Cashier cannot access.

2. Logic of the <> Relationships

Mandatory Sub-Processes: The use of the <> arrows from "Manage Books" and "Manage Users" indicates that they are base containers.

Process Integrity: It is assumed that if the user is in a session to "Manage Books," they will have to use one or more of the functions associated with it (Add, Delete, Update).

Navigation Logic: The inclusion of "Search book by Category" in the Manage Books use case indicates that searching for a book record is a required step before making any changes or deletions to the Book record.

3. Authentication Security and Session Management

Authentication: It is assumed that the "Login" label shown at the top of this page represents the specific system authorization required before any other functions on the system can be used. No user will be able to conduct any type of system-related function (like executing a sale) unless they have an active and authenticated session.

Session termination: The "Logout" button implies that there is an electronic record of distinct sessions of operation; each user should log out of the application after they are finished so that the workstation is properly secured and accessible only to authorized users. This is particularly critical in retail situations where multiple users are using the same workstation for processing transactions.

4. Scope of System Operations

Real-time inventory impact: The "Process Sale" use case should immediately reduce item quantities found in "View All Book Detail" to be the same as those recorded when they were sold.

Boundary of the System: The rectangle in a use case diagram defines the boundaries of the software system. There are other systems and applications (e.g., Payment Gateway, like a Credit Card, and Printing Devices) outside the boundaries of the retail application; however, these systems and their respective functions should be accounted for in the "Process Sale" use case even if they are not depicted as actor symbols.

Class Diagram

A Class Diagram is one of the simplest and most commonly used modelling tools available to users of the Unified Modelling Language (UML) (Fowler, 2003). A Class Diagram is used to perform Structural Modelling and also serve as a representation of the system's static view. The overall purpose of a Class Diagram is to illustrate the relationship between the various classes, attributes (data/properties) of each class within the system, operations (methods/behaviour) that can be performed by each class, and the associations that exist between all of the classes within the system (Booch et al., 2005).

In terms of Software Architecture, the primary purpose of a Class Diagram when compared to External (Behavioural) Diagrams (e.g., Use Case Diagrams) is to provide an Internal View of the System Architecture in detail. Additionally a Class Diagram has many other primary functions:

- Conceptual Modelling: This stage allows you to think of what your domain model will be for your problem space.

- Detailed Design: This stage allows you to provide the detailed specification of classes to be implemented in Object-Oriented Programming Languages (OOPs) also called shall provide by Object Management Group (OMG, 2017).

- Communication: A common visual language is needed for developers and other stakeholders to share an understanding of the structural portions of the software code base (Arlow & Neustadt, 2005).

Class Diagrams show many different types of relationships such as inheritance, association (i.e., the relationship between two or more Classes), aggregation & composition. All relationships that are defined in Class Diagrams indicate that these relationships exist in the nature of complex and interdependent object-oriented designs (Fowler, 2003).

The Class diagram below illustrates the proposed system for Little Book Haven Shop.

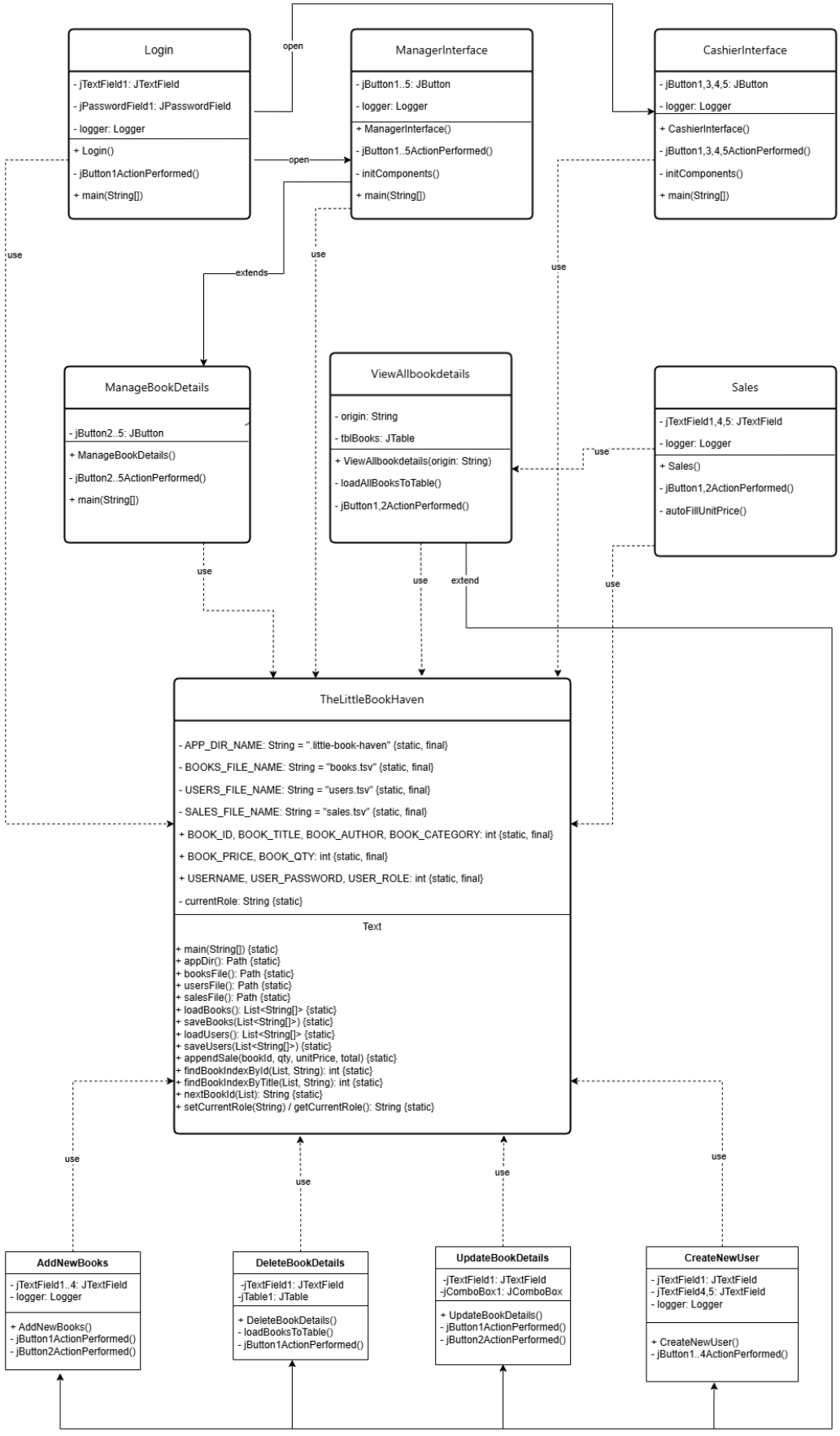

Login
- jTextField1: JTextField
- jPasswordField1: JPasswordField
- logger: Logger
+ Login()
- jButton1ActionPerformed()
+ main(String[])
ManagerInterface
- jButton1..5: JButton
- logger: Logger
+ ManagerInterface()
- jButton1..5ActionPerformed()
- initComponents()
+ main(String[])
CashierInterface
- jButton1,3,4,5: JButton
- logger: Logger
+ CashierInterface()
- jButton1,3,4,5ActionPerformed()
- initComponents()
+ main(String[])
open
open
use
use
use
extends
ManageBookDetails
- jButton2..5: JButton
+ ManageBookDetails()
- jButton2..5ActionPerformed()
+ main(String[])
ViewAllbookdetails
- origin: String
- tblBooks: JTable
+ ViewAllbookdetails(origin: String)
- loadAllBooksToTable()
- jButton1,2ActionPerformed()
Sales
- jTextField1,4,5: JTextField
- logger: Logger
+ Sales()
- jButton1,2ActionPerformed()
- autoFillUnitPrice()
use
use
use
use
extend
use
TheLittleBookHaven
- APP_DIR_NAME: String = ".little-book-haven" {static, final}
- BOOKS_FILE_NAME: String = "books.tsv" {static, final}
- USERS_FILE_NAME: String = "users.tsv" {static, final}
- SALES_FILE_NAME: String = "sales.tsv" {static, final}
+ BOOK_ID, BOOK_TITLE, BOOK_AUTHOR, BOOK_CATEGORY: int {static, final}
+ BOOK_PRICE, BOOK_QTY: int {static, final}
+ USERNAME, USER_PASSWORD, USER_ROLE: int {static, final}
- currentRole: String {static}
Text
+ main(String[]) {static}
+ appDir(): Path {static}
+ booksFile(): Path {static}
+ usersFile(): Path {static}
+ salesFile(): Path {static}
+ loadBooks(): List<String[]> {static}
+ saveBooks(List<String[]>) {static}
+ loadUsers(): List<String[]> {static}
+ saveUsers(List<String[]>) {static}
+ appendSale(bookId, qty, unitPrice, total) {static}
+ findBookIndexById(List, String): int {static}
+ findBookIndexByTitle(List, String): int {static}
+ nextBookId(List): String {static}
+ setCurrentRole(String) / getCurrentRole(): String {static}
use
use
use
use
AddNewBooks
- jTextField1..4: JTextField
- logger: Logger
+ AddNewBooks()
- jButton1ActionPerformed()
- jButton2ActionPerformed()
DeleteBookDetails
-jTextField1: JTextField
-jTable1: JTable
+ DeleteBookDetails()
- loadBooksToTable()
- jButton1ActionPerformed()
UpdateBookDetails
-jTextField1: JTextField
-jComboBox1: JComboBox
+ UpdateBookDetails()
- jButton1ActionPerformed()
- jButton2ActionPerformed()
CreateNewUser
- jTextField1: JTextField
- jTextField4,5: JTextField
- logger: Logger
+ CreateNewUser()
- jButton1..4ActionPerformed()

Figure 3

Key assumptions made after analysing the Use Case Diagram:

1. Architectural Pattern (Utility Class)

Centralized Handling of Data: This architecture uses a Utility Pattern architecture. Rather than having separate "Service" and/or "Repository" classes, all UI classes will use the TheLittleBookHaven class directly for data manipulation.

Stateless Utilities: The Little Book Haven will not be instantiated (no object state), and all of its methods and attributes will be static; therefore, it will be a global service to the entire system.

2. Storing and Persisting Data

File-Based Database: The constants BOOKS_FILE_NAME and USERS_FILE_NAME indicate that the system is storing data in Flat Files (.tsv or .csv) instead of using an RDBMS like MySQL.

Manual Indexing: The BOOK_ID, BOOK_TITLE, etc., are defined as integers, which means that the system is treating a row of data as an array -- the constants indicate which column position the constants correspond to in a row of data.

3. Interface and Lifecycle of Application:

Swing Framework is used as we are extending javax.swing.JFrame and so the application is assumed to be a Heavy-Client Desktop Application. Thus, we consider each of the functional Screens, i.e. Login, Sales, AddBooks, to be a separate Window.

Event Driven Interaction: The existence of jButtonActionPerformed methods indicates that the logical flow through the system is primarily initiated from User's clicks, following the standard Java AWT/Swing event model.

4. Dependency and Relationship Logic:

One Way Relationship/Dependency: Dashed arrows (<>) signify that UI classes are ‘Consumers’ of the logic that the Utility class provides. Therefore, it is assumed that the Utility class has no knowledge of the UI (loose coupling in only one direction).

Implied Authentication: The Login class is assumed to create a global state or "User Role" in the Utility Class, which will then be checked by the ManagerInterface and CashierInterface as to what functional capabilities are to be enabled or not enabled.

5. Error Handling and Maintenance

Centralized Logging: The classes assume that you have a practitioner debugging your application, as indicated by having a logger attribute in each class, which means your application logs to file instead of simply outputting to the console.

Validation Responsibility: The UI classes (AddNewBooks, UpdateBookDetails) will be responsible for validating the data (e.g., checking whether a price is valid input), before sending data to the utility class for saving.

Sequence Diagram.

The Unified Modelling Language (UML) has a specific part known as diagrams of sequencing, for the purpose of showing how a system has changes dynamically over time (Fowler 2019). They visualize how different types of activities/ processes will interact with one another, for example; sequencing through times to identify when an object (or actor) might interact with another object (or actor).

The analysis and design stage of a software application development process provides an opportunity for developers to demonstrate to stakeholders how the system will behave for each transaction or action taken (Pressman & Maxim 2020).In addition, the lifelines of the objects involved in that operation, and the messages that they may send amongst themselves and the order and timing of each message can help to discover potential bottlenecks or concurrency issues or what has been left out of sequence when analysing the activities of a particular use case (van Vliet 2008). Sequencing can provide a visual representation of how a conceptual model of a system relates to its implementation as a physical product.

The Sequence Diagram created for the proposed Little Book shop's operational environment is provided below.

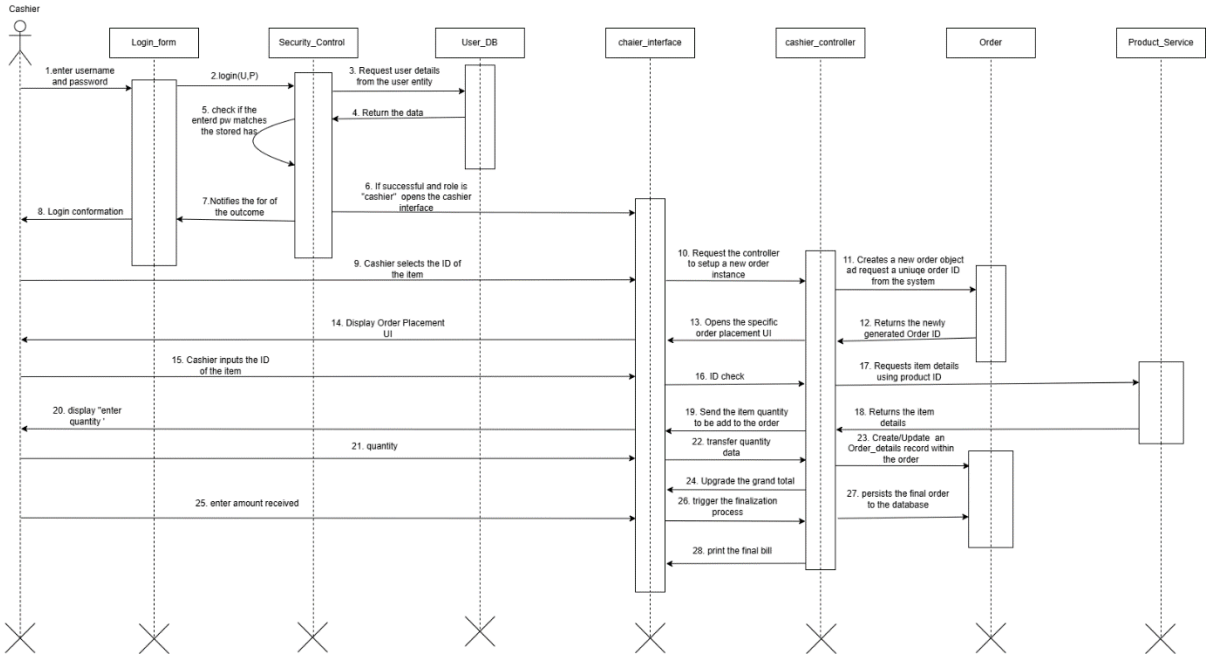


Figure 4

In addition to this Sequence Diagram (which captures the process of "Process Sale" and "Login"), the following assumptions form the basis of the system's dynamic behaviour and data-flow:

1. The system will use an architectural pattern that supports separation of concerns, such as a controller that is responsible for controlling the process of executing the cahier actions. The controller class will act as the coordinator of logic between the cahier interface (view) and order/product service (model/service layers).

2. Active orders will exist in the system while the cahier adds items; however, the order will not be persisted to the database (as an "Order" object) until the very end of the sequence (after Step 27).

3. The cahier will be redirected to his/her own cahier interface from the login process if he/she has the "Cashier" role (Security control will check for this role before granting access to the cahier interface prior to the Cashier logging in).

4. The cahiers' password will be hashed by the security control processes and will not be stored in plain-text. During the login process, security control will hash the entered password and the hashed value will be compared to the stored hashed value.

Task 02

This document provides details on how the Sweet Cupcake Shop Management System is being designed through the objectives and processes of developing an Object-Oriented (OO) Application, in accordance with Object, Class, Encapsulation, Abstraction, Polymorphism and Inheritance (all principles of OOP).

How is the system structured?

The architecture of the system follows a layered approach called the Controller-Service-Entity (C-S-E) model (from the basic Object and Class). This allows a unified approach to applying OOP principles, which are primarily based on the Single Responsibility Principle.

Object & Class

Class — An abstract representation of all objects that share the same properties or behaviour as an object is a class (Larman, 2004). In the Sweet Cupcake Shop Management System, the following three classes were used: User, Product, and Order.

Object — An actual instance of a class that is present in memory is an object. An example of this is when the Login-Page class creates an Object called Login-Window in memory.

Code for Class and Object: The Loginpage.java file has a class named Login page in which the main() method creates an Object of type Login Page. Therefore, the Loginpage.java file produces a Login-page object in memory when executed. This object is created when the Ok button is selected on the Login window. When the Login Page is executed by selecting an individual instance of the Login Page class, the login window is created.

```
151  public static void main(String args[]) {
152      try {
153          for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
154              if ("Nimbus".equals(info.getName())) {
155                  javax.swing.UIManager.setLookAndFeel(info.getClassName());
156                  break;
157              }
158          }
159      } catch (Exception ex) {
160          java.util.logging.Logger.getLogger(Login.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
161      }
162
163      java.awt.EventQueue.invokeLater(() -> {
164          new Login().setVisible(true);
165      });
166  }
```

Figure 5

Complex Logic Management and Data Protection

Encapsulation

Encapsulation means putting together all members of the class (i.e., all attributes/fields associated with that class and all methods used to manage those attributes/fields) with the intent of making them private attributes (these can only be accessed via the methods of that class), thereby ensuring the integrity of the data managed by the class (Budd, 2005).

How Each System Utilizes This Concept

- AccessDBConnection: Contains static final private variables for physical path of database (DATABASE_PATH) and for URL.
- Login Page Class: Encapsulates private variables jTextField1 and jPasswordField1 (GUI Components) so no other classes can directly change the content of these Input Fields.
- Source Code (Encapsulation - AccessDBConnection)

```
1
2   package the.little.book.haven;
3
4   import javax.swing.*;
5   import java.sql.*;
6
7   public class Connect {
8       public static Connection ConnectDB(){
9           try {
10              Connection conn = DriverManager.getConnection("jdbc:ucanaccess://database/book.accdb");
11              return conn;
12          } catch (Exception e) {
13              JOptionPane.showMessageDialog(null, "Error cannot connect to database: " + e.getMessage());
14              return null;
15          }
16      }
17  }
```

Figure 6

Abstraction

Abstracting out complex details of the services provided by other classes is the primary purpose of abstraction. Accessing the public method getConnection() on the class is primary public method that allows you to access a database by simply connecting to the service, without ever having to know how to create the connection.

In addition, because the AccessDBConnection class hides the database physical file path property and connection url property (DATABASE_PATH and URL), and the logic that creates the connection (using the JDBC Driver DriverManager.getConnection(URL) method), any other class only needs to be concerned with making the call to getConnection().

In short, each class can perform its function without needing to know how the AccessDBConnection class functions.

Specifically:

- It does not need to know what the path to the database file is.
- It does not need to know how to construct the URI for the connection.

- It does not need to know how to program the JDBC driver logic to connect to the database.

The services provided are abstracted by keeping the implementation details separate from the service provided itself.

```
package the.little.book.haven;

import javax.swing.*;
import java.sql.*;

public class Connect {
    public static Connection ConnectDB(){
        try {
            Connection conn = DriverManager.getConnection("jdbc:ucanaccess://database/book.accdb");
            return conn;
        } catch (Exception e) {
            JOptionPane.showMessageDialog(null, "Error cannot connect to database: " + e.getMessage());
            return null;
        }
    }
}
```

Figure 7

Polymorphism.

Polymorphism can be understood as performing the same function on a number of different objects in a variety of different manners. For example, after logging into the system, there is a determination made as to which Dashboard to display to the user (Stroustrup 2013).

Examples of implementing polymorphism

When the user has logged into the system, a Classification object (Manager or Cashier) will be used to generate and display a different Dashboard object (Class Instance) based on the user's Role.

- A dashboard is displayed using Polymorphism.
-

Object "Dashboard" will be created when the Role is "Manager" and created when Dashboard Admin Specific Interface will open.

```
String role = TheLittleBookHaven.findUserRole(username, password);
if (role != null && !role.isBlank()) {
    if (role.equalsIgnoreCase("Manager")) {
        TheLittleBookHaven.setCurrentRole("Manager");
        new ManagerInterface().setVisible(true);
    } else if (role.equalsIgnoreCase("Cashier")) {
        TheLittleBookHaven.setCurrentRole("Cashier");
        new CashierInterface().setVisible(true);
```

Figure 8

Object "Cashier Dashboard" will be created when the role is "Cashier" and created when Cashier Specific Interface will open.

```
if (role.equalsIgnoreCase("Manager")) {
    TheLittleBookHaven.setCurrentRole("Manager");
    new ManagerInterface().setVisible(true);
} else if (role.equalsIgnoreCase("Cashier")) {
    TheLittleBookHaven.setCurrentRole("Cashier");
    new CashierInterface().setVisible(true);
} else {
    JOptionPane.showMessageDialog(this, "Unknown role: " + role
```

Figure 9

Polymorphic implementations within this example include the creation of an Admin Dashboard object for the Admin role, and subsequently the Open for Admin Dashboard informational message is provided to the user.

Similarly, for the Cashier role, the System generates and displays a Cashier Dashboard object (sometimes referred to as cDashboard) and the Cashier Dashboard informational message will be provided to the user.

Inheritance.

Inheritance is one of the essential elements of Object-Oriented Programming (OOP) that provides software engineers with a way to encapsulate new class implementations (also called child or subclass) while retaining all the attributes (a.k.a. fields) and the previous methods (functions or "code") of existing class implementations (also called parent or superclass.)

The main benefits of using inheritance while developing software include:

1. Reuse of Code: Because of the ability to reuse existing code, developers do not need to write the code to create a window or a button for the window or to minimize the window again (e.g., the code for creating windows already exists in the JFrame Class, allowing a Sales Class to use this code when creating its own windows);

2. Override of Methods: The ability to implement existing methods defined in the parent class differently at runtime by a child (subclass) allows runtime polymorphism (i.e., different implementations of a method being run at run time)

3. Logical Structure: Logically structuring classes with the inheritance relationship (e.g., creating subclasses of User class like Manager and Cashier; therefore, both will be defined as Users)
4. Easier to Maintain: A parent class can be changed once, and therefore, all children classes will be automatically updated when changes are made to the parent class, therefore, reducing the amount of time for updating a single item within the code base.

```
21          */
22          public CashierInterface() {
23              initComponents();
24              this.setSize(450, 350);
25          }
26
```

Figure 10

Task 03

- User Manual for “THE LITTLE HAVEN BOOK SHOP” System

The following is an explanation of how to perform these tasks:

Log into the system by running the project in NetBeans IDE. You may do this by pressing the button labelled Run Project or by using the keyboard shortcut Shift + F6.

Figure 11

All users will see the same login screen once the application has been installed; however, the system will redirect users to their respective dashboards based on their role (polymorphism).

Instructions:

- User Name: Manager
  Cashier
- Password: 123 (Manager)
  456 (Cashier)
- Please select your desired option from the list of available options provided by clicking on the Login button.

Dashboard By Role:

Admin: After a successful login, you will be redirected to the Administration Dashboard which allows access to functions like Product Management.

Figure 13

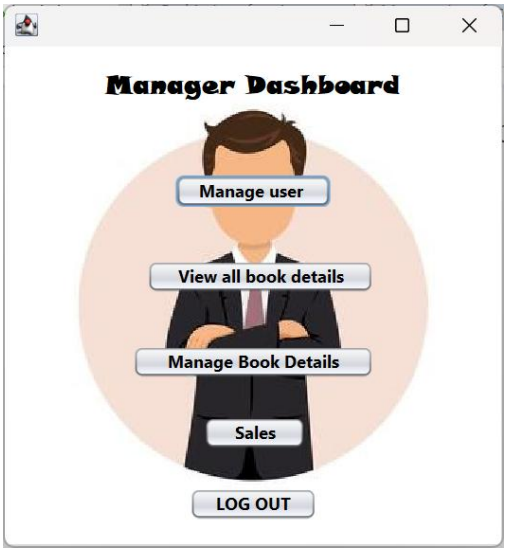


Figure 12

The Cashier Dashboard interface will be displayed after successful login into the Cashier's workstation - thereby allowing full accessibility to functionality associated within the Cashier application such as: Order Entry.

Figure 15

Figure 14

To log out of the system, you need to press the Logout button and confirm that you want to log out of the system.

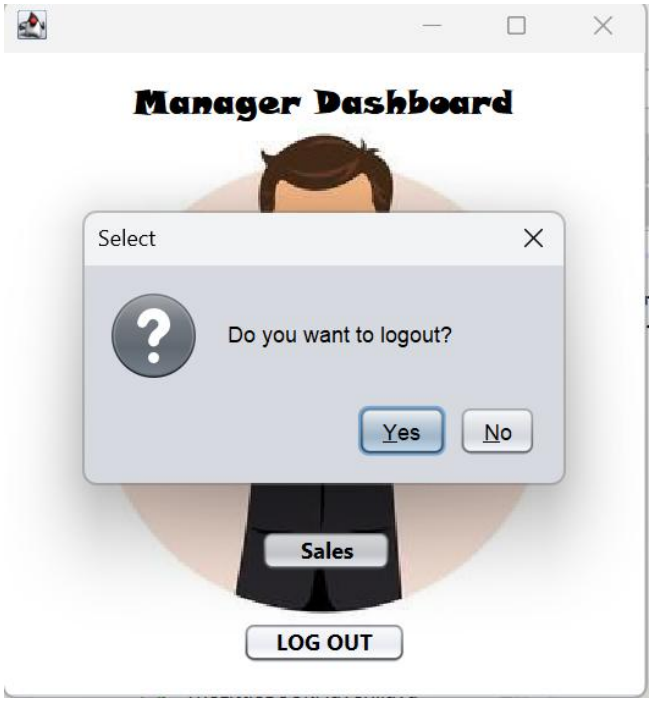
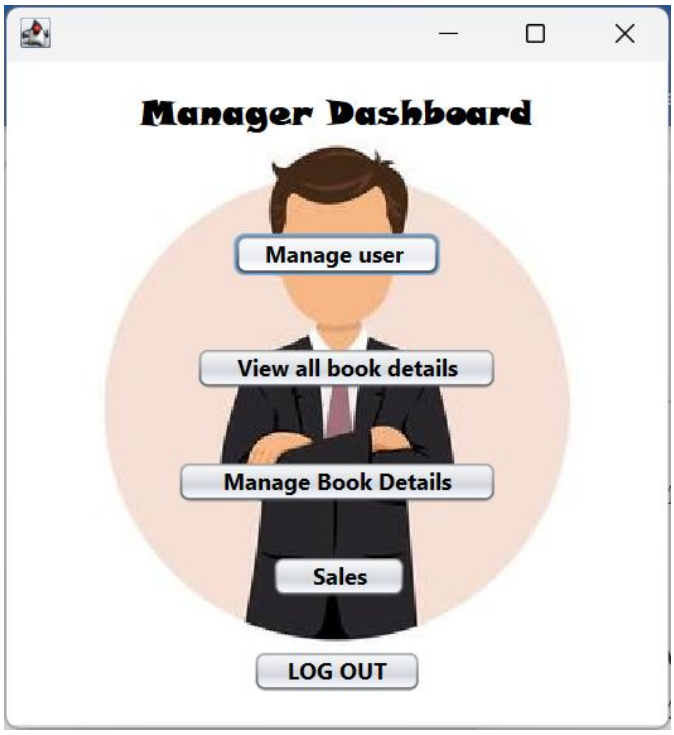


Figure 16

Manage Users Process

The Manager has access to create a new user account.



Click “Manage User ” button

Figure 17

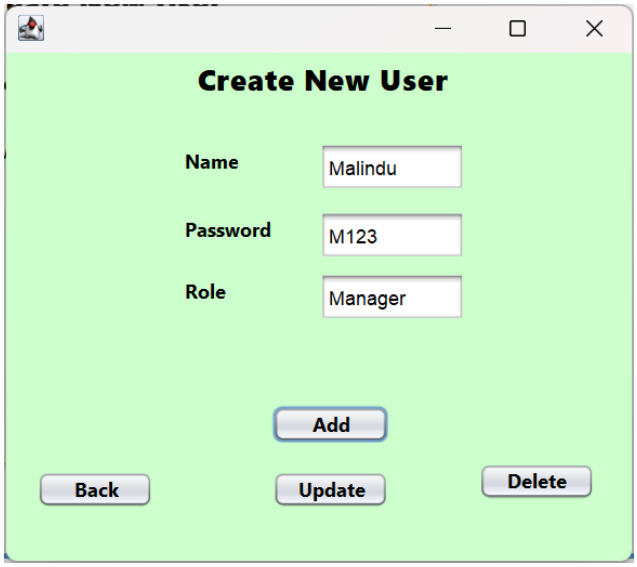


Figure 19

Complete the necessary Details as required by clicking on the Add button.

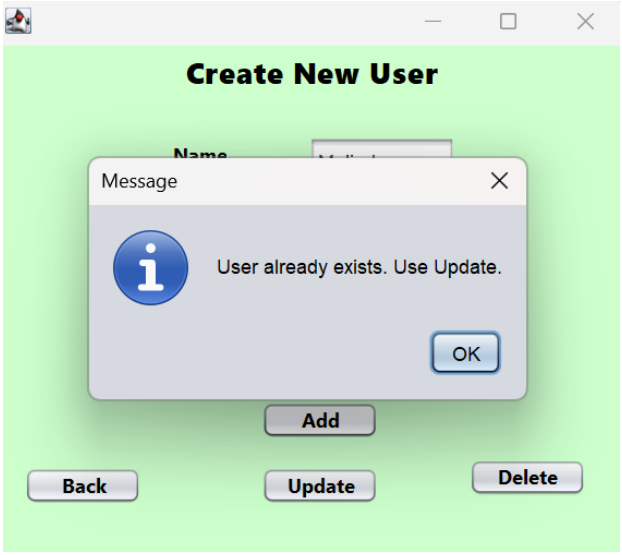


Figure 20

Click OK

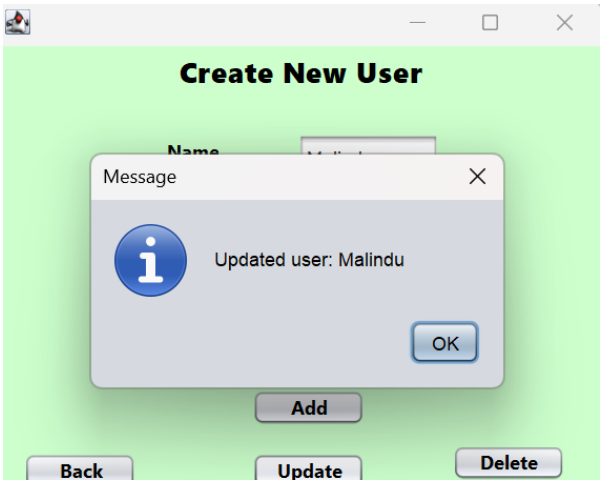
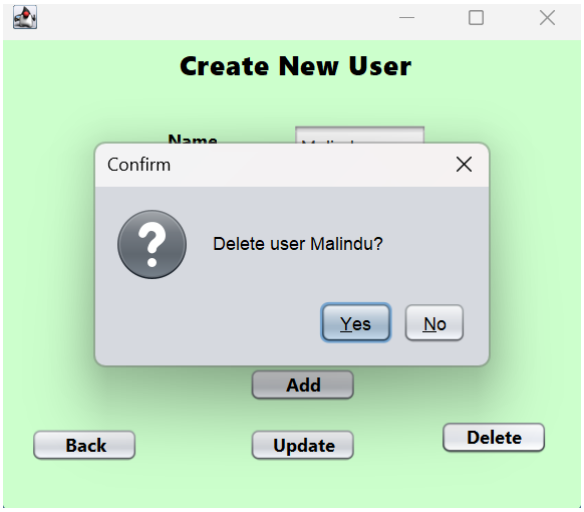


Figure 18

Update a User. click Update button



Delete a User. Click “Delete” button

Figure 21

To add new User Object account information into the database, the createUser(User user) method in the Admin Controller class retrieves the details for that User and adds them to the database as well.

history | Users

ID	name	password	role	Click to Add
41	admin	123	Manager	
42	cashier1	123	Cashier	
43	Cashier2	456	Cashier	
44	sasi	232323	Cashier	
45	sasi11	222	manager	
46	Malindu	M123	Manager	
(New)				

Figure 22

View Book Details Process

The View Book Details interface is valid for both Manager and Cashier.

Click “View all book details” button.

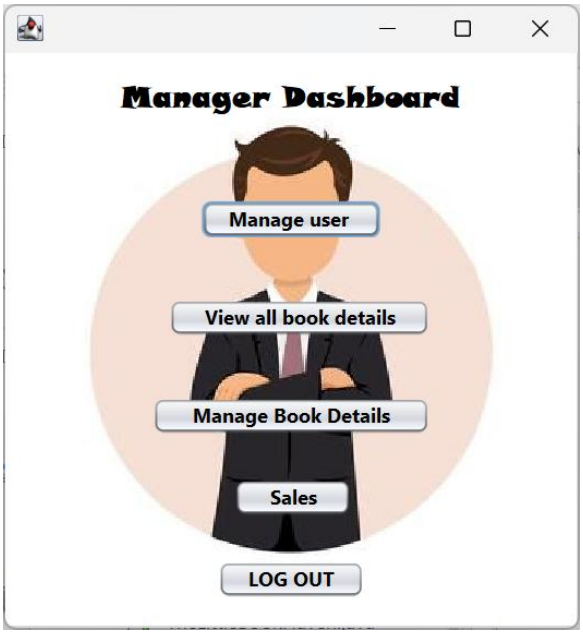


Figure 24

Figure 23

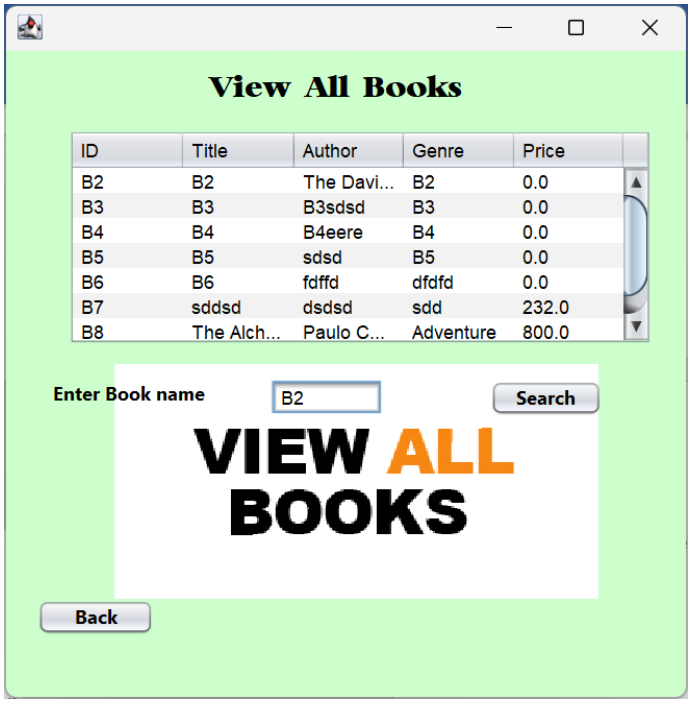
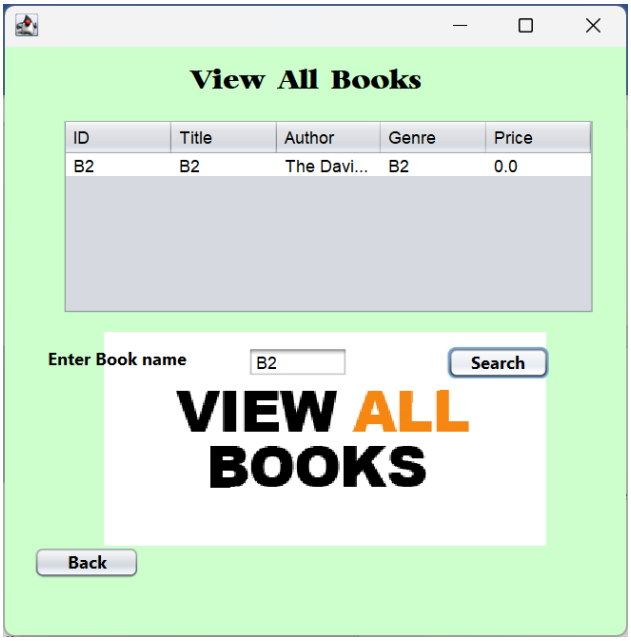


Figure 25

Enter Book name and then click Search button.



The name of the book you searched for appears.

Figure 26

Manage Books Details Process

Manager/Cashier interface click the Manage Book Details bouton.

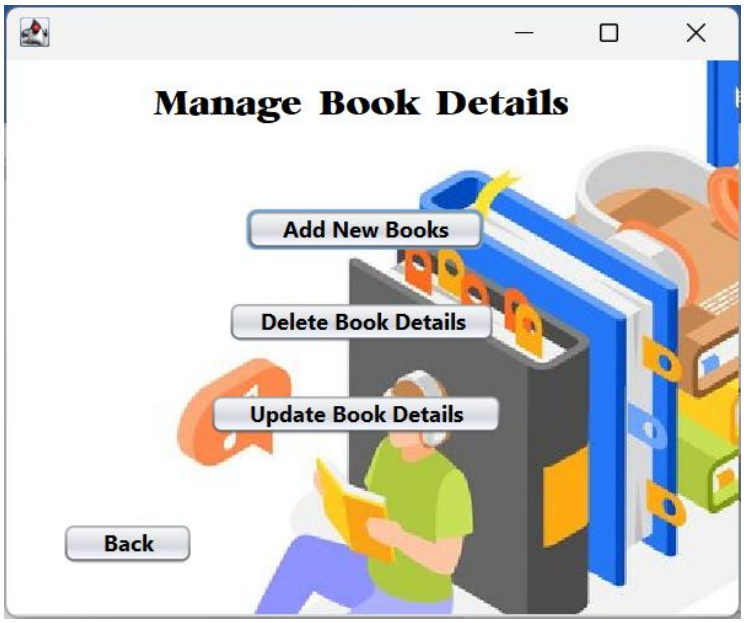


Figure 27

Now let's see how to add a new book.

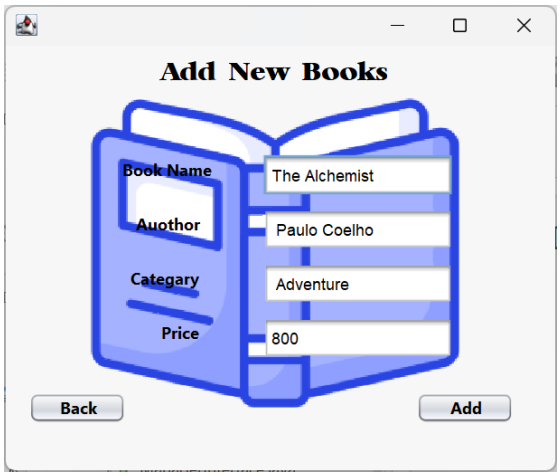


Figure 29

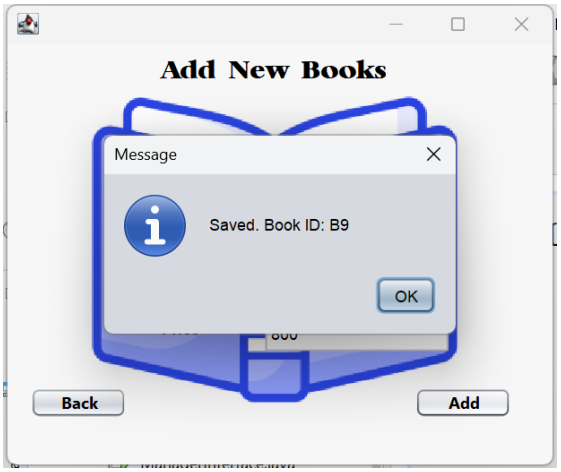


Figure 28

Click ok button

Goes back to the Manage Book details interface.

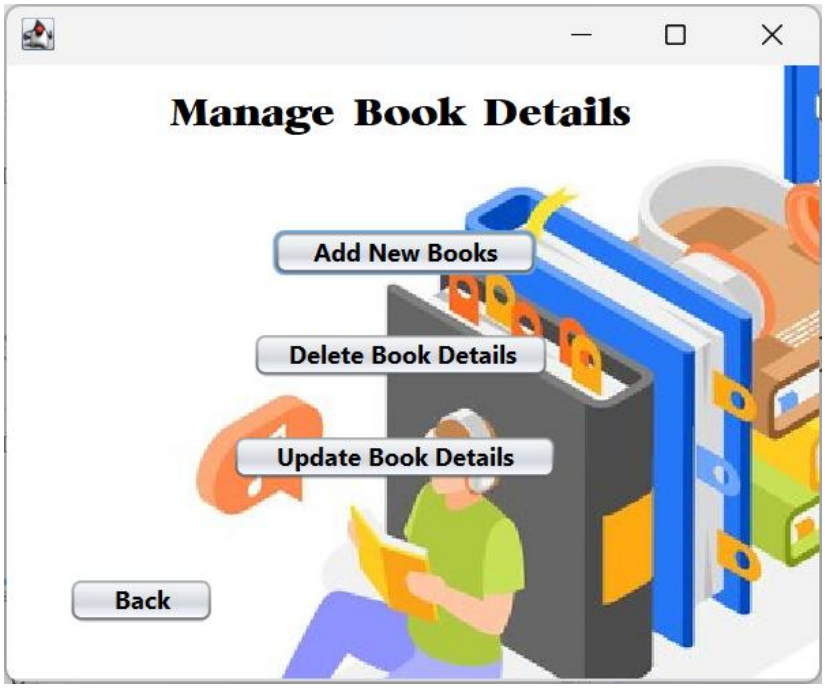


Figure 30

Next, let's see how to delete a book.

Click Delete Book Details Button

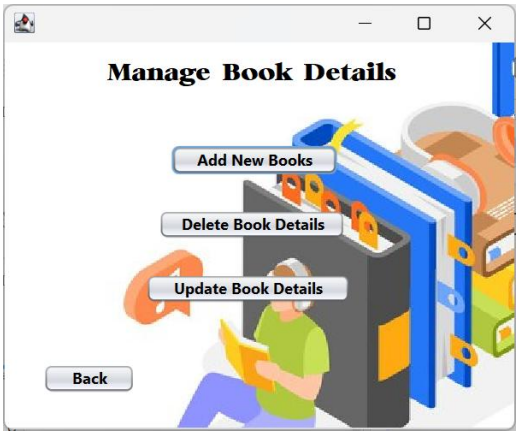


Figure 31

Enter Book ID

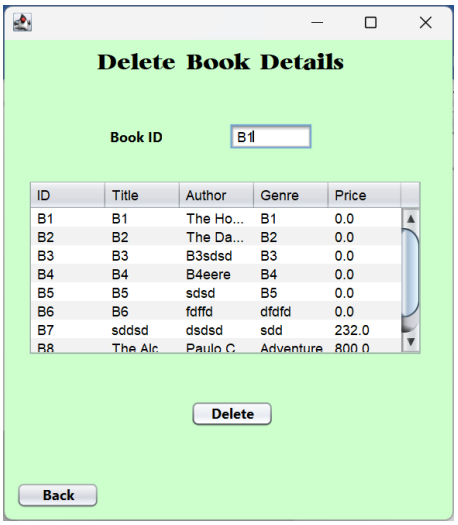


Figure 32

Click “Yes” Button

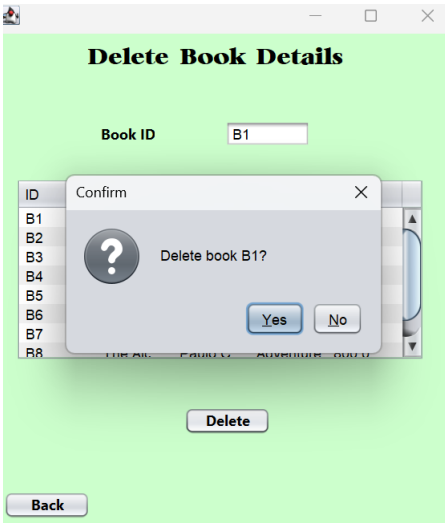


Figure 33

Click “OK” Button

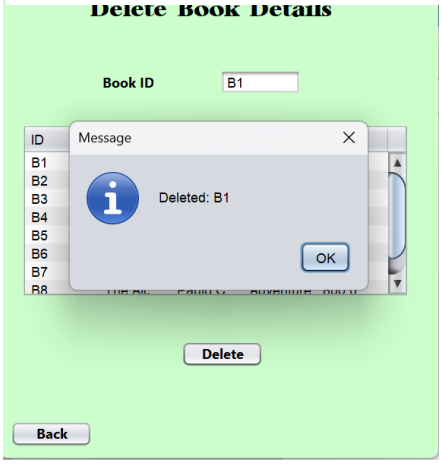


Figure 34

Finally, let's see how to update a book.

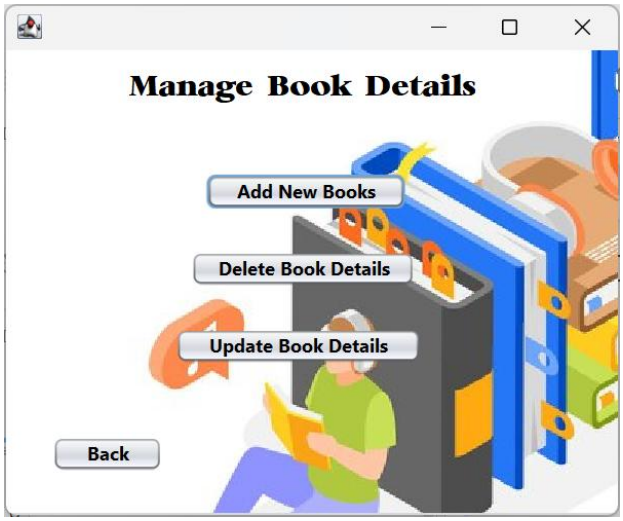


Figure 35

Click Update Book Details Button

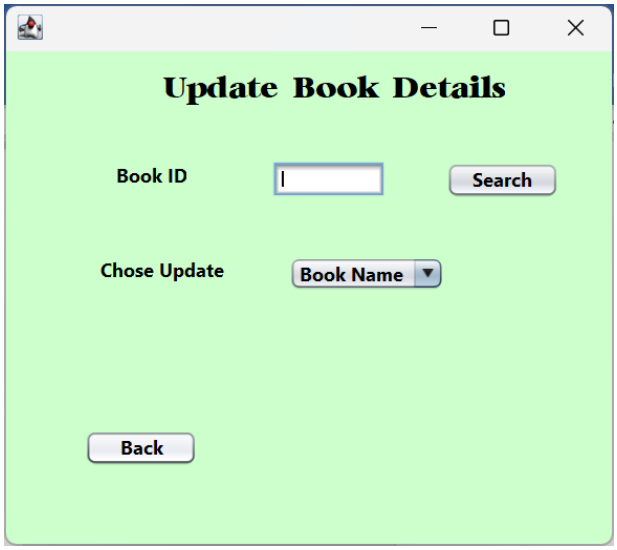
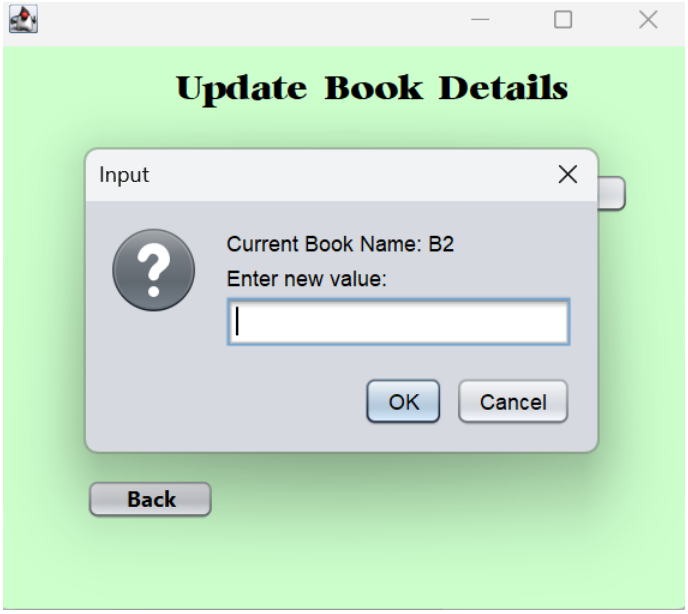


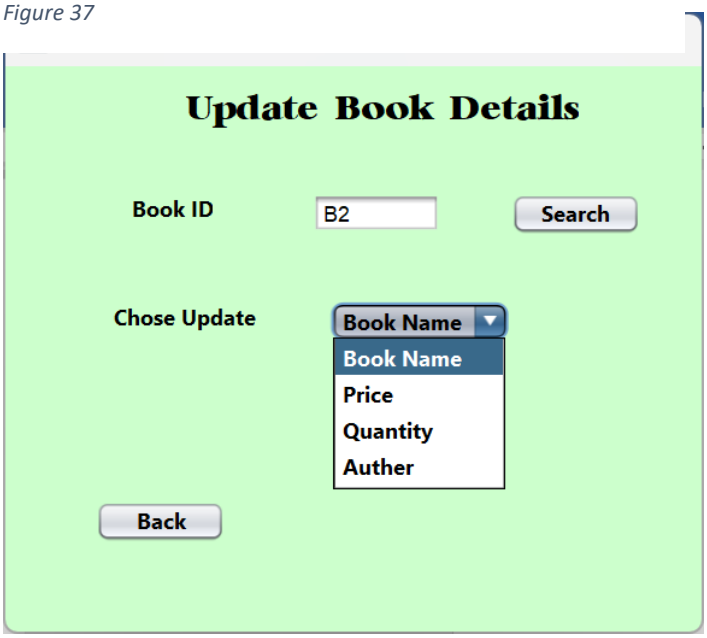
Figure 36

Enter Book ID and Click Search button.



Enter new value

Figure 37

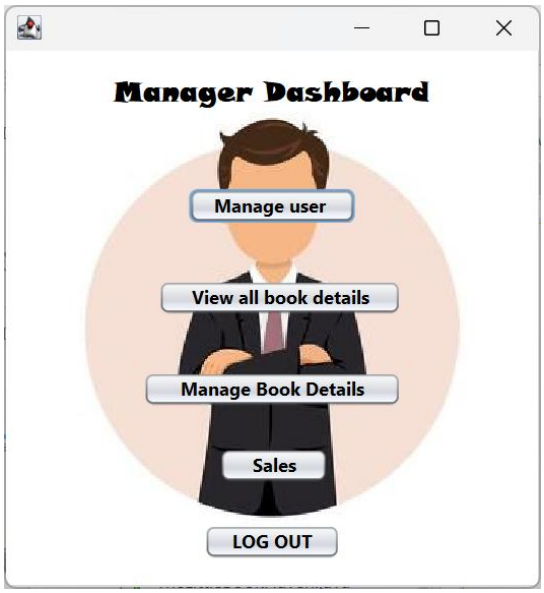


Chose Update title

Figure 38

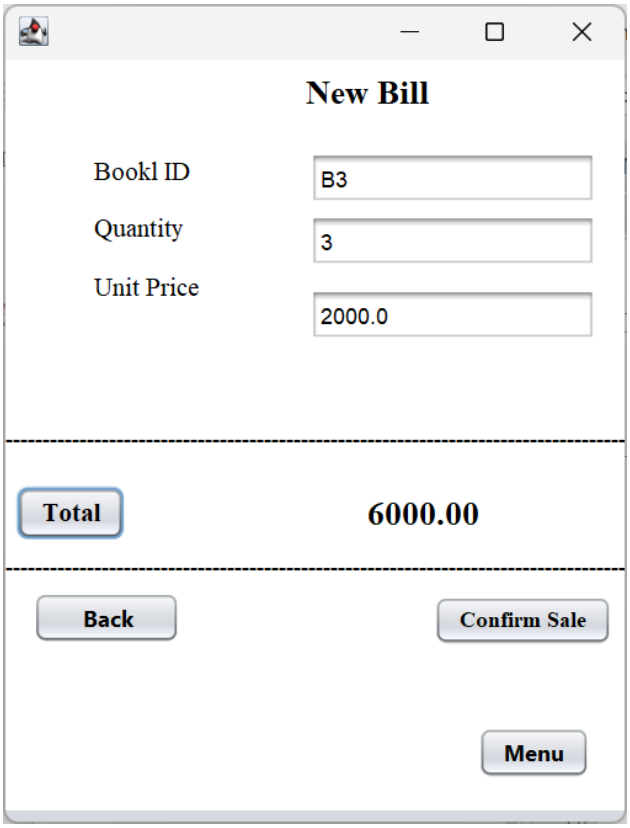
Salles Process

Manager/Cashier interface click the Sales bouton.



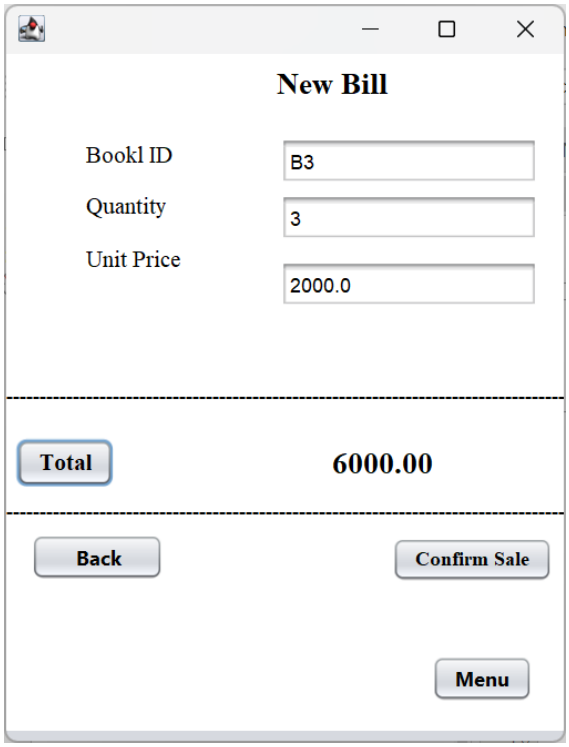
Click “Sales” Button

Figure 39



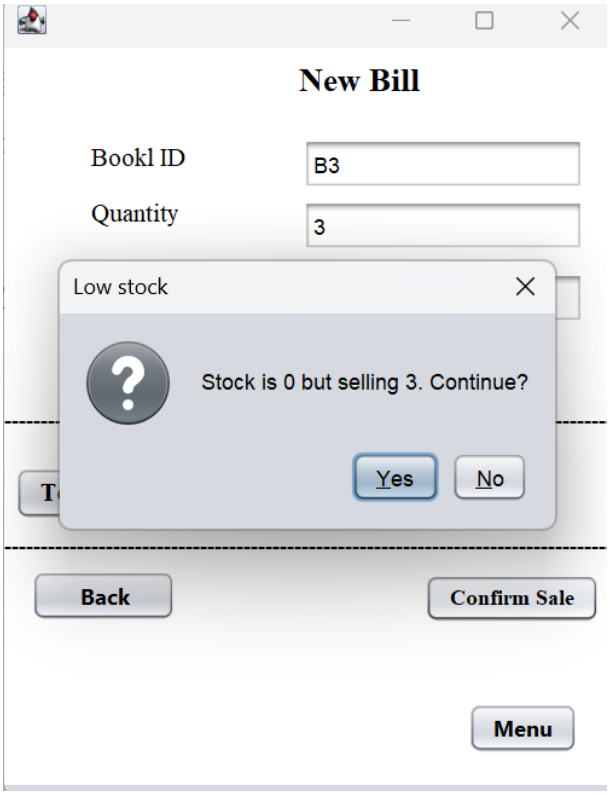
Fill the Details

Figure 40



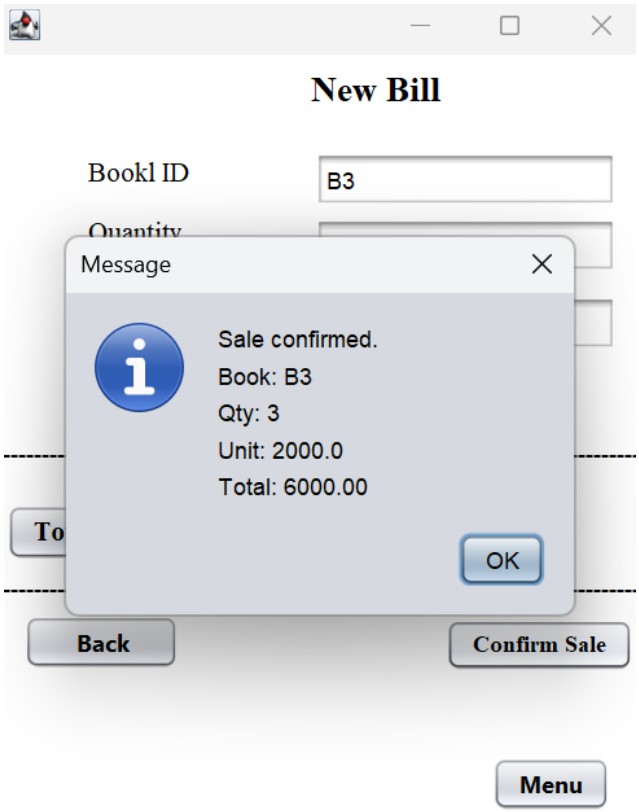
Then Click “Total” button

Figure 41



Then Click “Conform Sale” button

Figure 42



Click “OK”

Figure 43

Once your shift has concluded, you should log out of the system securely by clicking on the “Menu” option to log out of the system.

Figure 44

There are multiple levels in the system they are based on an Object Oriented (OO) structure that allows a high level of flexibility and long-term sustainability of the Little Book Haven Shop Management System Development Process.

Project

The Little Book Haven.rar

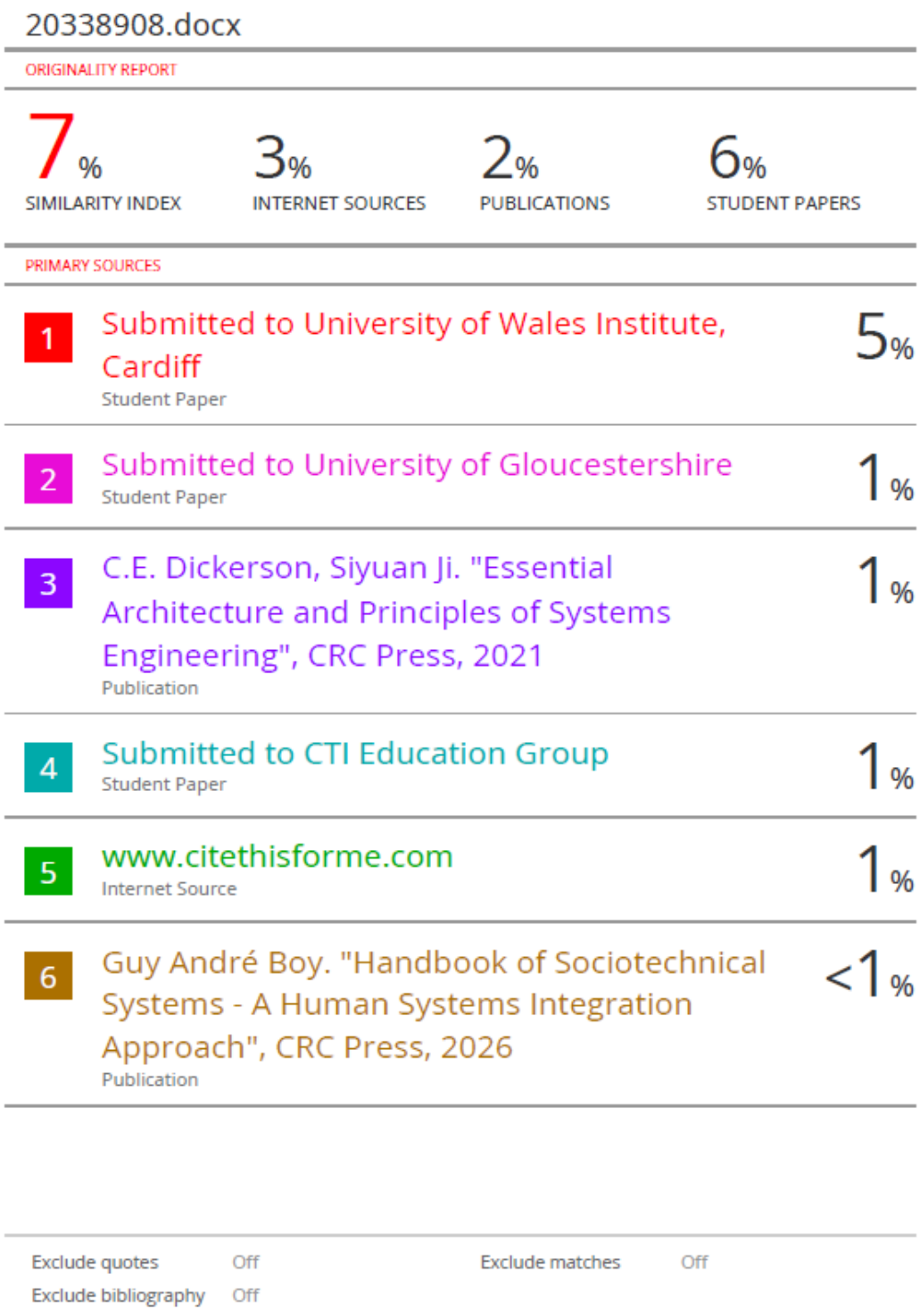
Conclusion.

In conclusion, this automated system has been developed to increase the efficiency of daily cash register and inventory operations, and to provide a professional and dependable manner in which to help improve the overall level of customer service at The Little Book Haven. The design of this solution has been completed according to the Object Oriented Design (OOD) principles of modularity, scalability, and maintainability. In addition, the Unit Testing process will provide an additional level of assurance regarding the reliability of the automated system. Lastly, by using OOD concepts of encapsulation, inheritance, and polymorphism the system is expected to be at a very high level of software engineering standards. The incorporation of File I/O (File Input/Output) provides a method to have permanent storage and protection of inventory and user information without complex database systems.

Reference.

- **Burd, B. (2012).** *Beginning Programming with Java for Dummies*. 3rd ed. New Jersey: John Wiley & Sons.
- **Fowler, M. (2004).** *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd ed. Boston: Addison-Wesley Professional.
- **GeeksforGeeks (2023).** *Object Oriented Programming (OOPs) Concept in Java*. [online] Available at: https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/ [Accessed 21 March 2026].
- **Murach, J. (2017).** *Murach's Java Programming*. 5th ed. California: Mike Murach & Associates.
- **Oracle (n.d.).** *Java Documentation*. [online] Available at: https://docs.oracle.com/en/java/ [Accessed 21 March 2026].
- **Preece, J., Rogers, Y. and Sharp, H. (2015).** *Interaction Design: Beyond Human-Computer Interaction*. 4th ed. New York: Wiley.
- **Tutorialspoint (2020).** *Java Tutorial*. [online] Available at: https://www.tutorialspoint.com/java/index.htm [Accessed 21 March 2026].
- **Visual Paradigm (n.d.).** *What is Unified Modeling Language (UML)?*. [online] Available at: https://www.visual-paradigm.com/guide/uml/what-is-uml/ [Accessed 21 March 2026].

Appendix



20338908.docx

ORIGINALITY REPORT

7%	3%	2%	6%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to University of Wales Institute, Cardiff Student Paper	5%
2	Submitted to University of Gloucestershire Student Paper	1%
3	C.E. Dickerson, Siyuan Ji. "Essential Architecture and Principles of Systems Engineering", CRC Press, 2021 Publication	1%
4	Submitted to CTI Education Group Student Paper	1%
5	www.citethisforme.com Internet Source	1%
6	Guy André Boy. "Handbook of Sociotechnical Systems - A Human Systems Integration Approach", CRC Press, 2026 Publication	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		